Data & AI 6 Reinforcement Learning Project 2025-2026

Alexander Michielsen

Overview

In this final project, you will work in groups of two to experiment with Deep Reinforcement Learning (DRL) algorithms using **Stable Baselines 3**. You will perform reward engineering by creating a custom reward function, implementing an additional extension, and ensuring thorough logging, visualization, and analysis of your results.

Objectives

- **Training**: Properly implement and train DRL algorithms using Stable Baselines 3.
- **Reward Engineering**: Design and implement a custom reward function for your specific problem.
- **Algorithm Comparison**: Implement one of the additional extensions and compare the performance.
- **Logging and Analysis**: Thoroughly log, visualize, and analyze your results.

Project Requirements

Note: Please take into account that part of your final grade will be based on the oral examination of the project. Remember that you can use Google Colab for this project. This will allow you to use a GPU for training your models.

1. Baseline

- Choose a **simple** environment from the Gymnasium library (e.g., MountainCarContinuous-v0, Pendulum-v1, MountainCar-v0, LunarLander-v2 (more advanced), CartPoleSwingUp-v0 (requires additional package)). Don't choose one we covered during the lectures. Also, don't go for a very hard one from the start, only consider more challenging environments once you get everything up and running.
- Frame the problem and define the goals.
- Utilize **Stable Baselines 3** to access pre-implemented DRL algorithms (e.g., DQN, SAC, PPO, A2C, TRPO).
- Train an appropriate DRL algorithm using the default reward function as a baseline. Use three runs for robustness and report their curves, mean, std for all relevant metrics (e.g., average return, convergence speed, problem-specific metrics)
- Record logs and visualize training performance (e.g., using TensorBoard).

2. Reward Engineering

- Analyze the default reward structure of the chosen environment. Include this analysis in your report.
- Design a custom reward function that better aligns with the desired behavior of the agent. **Justify your design choices**.
- Implement the custom reward function within the environment.
- Retrain the same DRL algorithm using your custom reward function.

- Compare the performance of the agent trained with the custom reward function to the baseline (e.g., average return, convergence speed, problem-specific metrics). Be thorough, you might need more metrics than these (think about what you would tell a potential client if you were to sell your solution).
- Use multiple (at least 3) runs for training to ensure robustness of results (properly aggregate the results (mean, std, training graphs overlayed)).

3. Algorithm Comparison

Choose one (or two as a bonus) of the following extensions to implement and compare:

- **Different DRL Algorithms**: Train one different DRL algorithm in the same environment with your custom reward function. Compare performance to the original algorithm in the custom environment (using the same metrics as the previous parts). Use three runs to ensure robustness of results.
- **Hyperparameter Tuning**: Choose one important hyperparameter (e.g., learning rate, batch size, gamma, entropy coefficient, ...). Try at least three sensible values for the hyperparameter. Analyze the impact of hyperparameter choices on performance (using the same metrics as the previous parts). Compare the different hyperparameter configurations with each other and with the baseline. (One run/configuration is sufficient)
- **Exploration Strategies** (a bit more challenging): Implement a different exploration strategy (e.g., ϵ-greedy, Boltzmann) and compare the impact on learning performance compared to the baseline (using the same metrics as the previous parts and using three runs again). You have to figure out what the default exploration is for your chosen algorithm. Make sure the exploration strategy you choose is compatible with your algorithm.

General Advice

- Use Tensorboard (or similar) to visualize your training curves.
- Use checkpoints to save your models during training.
- You should always evaluate your agents without exploration (*deterministic=True*)!
- If possible, run your training on a GPU to speed up the process (you can use Google Colab).
- RL training can take a while, so plan accordingly and try to run multiple experiments in parallel if possible.
- Make sure you properly compare the different implementations, this is highly important.

Deliverables

a. **Written Report** Combine your findings into a comprehensive report (**maximum 6 pages** (excluding the cover page)), including clear and readable figures, additional pages will **not be considered** during grading). Use the following structure (**Use the template provided on Canvas**):

   - **Introduction** (1 paragraph)
     - Overview of the project and objectives.
   - **Methodology** (max. 1 page)
     - Description of the DRL algorithm(s) used.
     - Custom reward function design and rationale.
     - Details of the additional extension implemented.
   - **Results and Discussion**
     - Analysis of default reward performance.
     - Training curves and performance metrics.
     - Comparative analysis of algorithms

        * Default reward vs. custom reward
        * Custom reward with and without the extension
      - Challenges faced and solutions implemented.
    - **Conclusion** (1/2 paragraphs)
      - Summary of findings and contributions.
      - Recommendations for future work.

b. **Code Submission**

   IMPORTANT: You must follow the 'provided folder structure and scripts. Do not change any names (of files or variables).

   Submit all code used for the project, including:

   - Scripts for training and evaluation.
   - Implementation of the custom reward function.
   - Code for the additional extension.
   - Instructions for running the code (e.g., README file).
   - Ensure code is well-documented and organized for readability.
   - Only include the relevant files in your submission

c. **Oral Defense** During the exam period, you will get a couple of questions about your submitted work. Be prepared to explain your design choices, challenges faced, and insights gained from the project.

Additional Resources

- **Gymnasium Documentation**: https://gymnasium.farama.org/
- **Stable Baselines 3 Documentation**: https://stable-baselines3.readthedocs.io
- **DRL Algorithms**: Review literature on the algorithms you choose to understand their mechanisms and suitable applications.

Submission Guidelines

- **Deadline**: *29 December 2025, 23:59*.
- **Submission Method**: Upload your Python scripts and report as a single ZIP file to Canvas

Good Luck!